

CloudSync Report

NAKUL KADAM

MockAPI Project link :

<https://mockapi.io/projects/6a398bb564a2d82692241c72>

logs (Entries saved here) :

<https://6a398bb564a2d82692241c71.mockapi.io/logs>

8 States of FSM :

1. STARTUP :

Initial state for the system which runs once at the start. The LCD display & variables get initialized. System is put in STATE_IDLE after the initialization.

2. STATE_IDLE :

The default state. The system waits for digit keys to be pressed on the keypad. Once pressed, jumps to ENTERING.

3. ENTERING :

Reads the keys pressed and runs the respective code for it.
System jumps to GRANTED if correct ID entered; else, jumps to DENIED.
Also checks timeout for key pressing.

4. GRANTED :

This state is reached if correct ID was entered. Runs code for finding the user in database and sends the log attempt to API Server. Clears all the display and puts the system back into STATE_IDLE.

5. DENIED :

This state is reached if the entry was failed.
Puts system into LOCKOUT state if this was the third consecutive failed entry.
If not, puts system back to STATE_IDLE.

6. LOCKOUT :

The system remains in a condition of visual lockout state for 15s. The LEDs are blinked for every alternate 300ms by calling handleLockoutBlink(); and the keypad is disabled for entry during the lockout.
After 15s, the system goes back to STATE_IDLE.

Pushing Hardware override button during lockout enables EMERGENCY state; which runs the emergency sequence code and allows emergency entry.

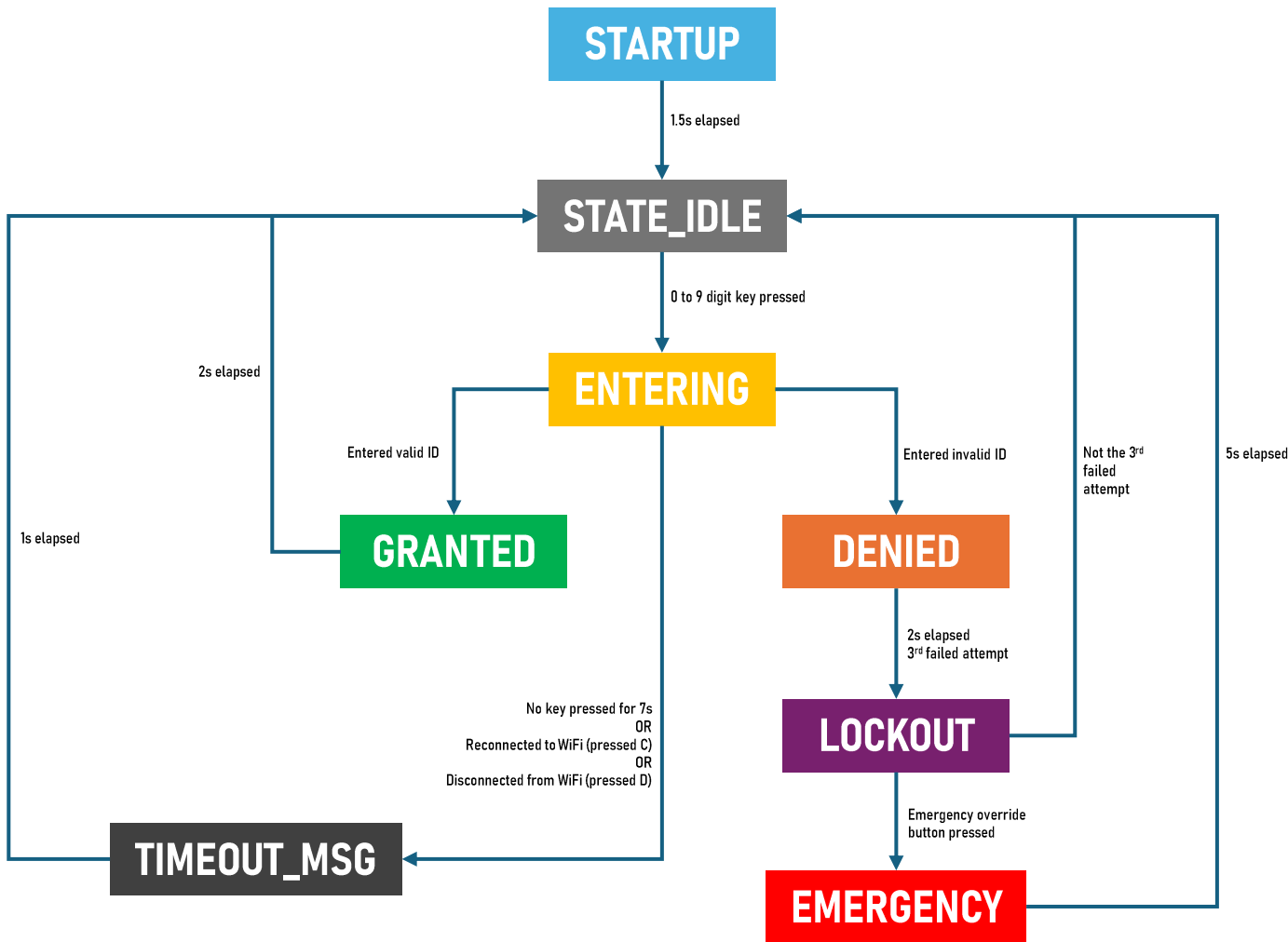
7. EMERGENCY :

Displays the message for override. Then clears the inputs and puts state back to STATE_IDLE.

8.TIMEOUT_MSG :

Displays the message for timeout (Only if no key pressed for 7s).
Puts state back to STATE_IDLE.

State transitions flowchart :



14 Functions in the code :

1. void showPrompt() :

Clears the previous display and displays the text “Enter 4-Digit ID”.

2. void clearLEDs() :

Turns off all three LEDs.

3. void updateMaskedInput() :

The input string is updated & stored as enteredID. This function clears the bottom line of LCD and uses a for loop to replace each input digit with “*”.

4. int findPeep(String id) :

Context : Peep is a custom struct which stores the valid users’ role & id.

Traverses the id among all the users peeps. If id matches, return index of the user; else, return -1 (error).

5. void handleKeypad() :

Context : Button C is assigned to Connect to WiFi. Button D is assigned to Disconnect from WiFi. Button * is used as backspace. Button # is used as an enter for the id.
manualWiFiOff variable used as a digital button (true when WiFi turned off, false when WiFi turned on).

Every key pressed is stored as “key” using Keypad.getKey(). If key isn’t among the options we use on the hardware, return (stay in ENTERING mode).

Upon pressing “C”... Display “WiFi already”, “Connected!” if WiFi is on. Else, Display “Connecting...”, reset manualWiFiOff and reconnect to WiFi by calling connectWiFi().
Reset the enteredID to “”.

Upon pressing “D”... Displays “WiFi already”, “Disconnected!” if WiFi is off. return to ENTERING immediately.
Else, set manualWiFiOff and disconnect WiFi by calling WiFi.disconnect(). Display “WiFi Disabled”, “Manually”.
Reset the enteredID to “”.

For number keys being entered, update lastKeyTime to millis() and put the system in ENTERING mode.
If key is a number, extend enteredID only till 4th digit is reached. Mask the digits by calling updateMaskedInput().
Else if * is pressed, ensure length of enteredID is >0. Then delete the recent digit by using .remove(). Mask the remaining String by calling updateMaskedInput().
Else if # is pressed, store the returned value from findPeep into a variable idx. Reset enteredID to “”.
Check if idx>0 (means valid user found), get the role of the user with peeps[idx].role and call enterGrantedState.
If valid user not found, increment count of failed attempts and call enterDeniedState().

6. void handleTimeout() :

Firstly check whether enteredID is empty string or not. If yes, return (Stay in STATE_IDLE).
If 7s passed since last key pressed, reset enteredID to “” and put state in TIMEOUT_MSG.

7. void enterGrantedState(String role) :

Context : LogEntry is the custom offline queue structure. Each entry is distinguished by three information : role, timestamp and granted (true for successful entry).
entry is the variable of LogEntry type. It holds the three information for the user during every attempt.

Store the three information in the blank entry.
If entry is made when WiFi is connected (checked using !manualWiFiOff && WiFi.status() == WL_CONNECTED), run postLog(entry). If postLog(entry) returns false, make the current entry in offline queue by calling queueLog(entry).
Else, means WiFi disconnected, offline queue the entry by calling queueLog(entry).

Display message “Welcome”, “role” and light the respective LED & play the respective buzzer tune.
Put the system to GRANTED state.

8. void enterDeniedState() :

Displays the message “Access Denied”, “Attempt failedAttempts”. Play the buzzer tone for Denied.
Put system back to DENIED state.

9. void handleLockoutBlink() :

Context : blinkTimer is timer for counting the 300ms LED on/off during lockout; initializes to 0ms.
blinkState is the LED input which we flicker during the lockout; initializes to false.

Run the sequence during the 15s of LOCKOUT state.
Wait 300ms by using millis() - blinkTimer > 300, update the blinkTimer to millis(), invert the blinkState and assign LED values to blinkState.

This functions makes the LED flicker every 300ms during the 15s lockout.

10. void emergencySequence() :

Display the message “Emergency Access”, “Dir. Nick Fury”.

11. void flushQueue() :

Context : sent is the local variable of this function that is a counter for successful entries made in API. Reset to 0 after each flush.

Return to STATE_IDLE if either WiFi disconnected or offline queue is empty (checked using (manualWiFiOff || WiFi.status()!=WL_CONNECTED || queueCount==0)).

Use a for loop to flush the queue by calling postLog() in the loop. If postLog returns true, increment sent. Else, exit the flush loop.

Compare sent & queueCount. If both equal at the end of flush loop, means all entries were logged. Reset queueCount. Else, run a for loop to shift all the unsent entries at the top of the offline queue. After the shifting, update queueCount to the current value as queueCount-sent.

12. void connectWiFi()

Context : WIFI_SSID is the network name. (“Wokwi-GUEST”).
WIFI_PASS is the network password. (“” for our project).

Runs command WiFi.begin(WIFI_SSID, WIFI_PASS) which uses .begin() to connect to WIFI_SSID network by taking password WIFI_PASS.

13. bool postLog(LogEntry &entry) :

Context : SERVER_URL is the custom variable that stores the address of MockAPI website (<https://6a398bb564a2d82692241c71.mockapi.io/logs>).

In the project’s HTTP, ESP32 is the Client and MockAPI is the Server.

body is a dummy String created which will later hold the text of JSON-formatted user information.

code is an int variable that stores the response from the Server (true if successful entry. Else, false).

HTTP returns 3-digit values as response. 200 or 201 means successful log in API. Any other value means failed log.

StaticJsonDocument is used as it allocates on the stack; not in heap. This prevents heap allocation & deallocation; which further eliminates issues of heap fragmentation.

entry is referenced using ‘&’ to avoid copying of the entry (String variables) that might occupy RAM.

If WiFi is disconnected (checked using (manualWiFiOff || WiFi.status() != WL_CONNECTED)), return false and exit the function.

Create HTTP variable http. Use .begin(SERVER_URL) to start the communication with server.

Use .addHeader("Content-Type", "application/json") to specify that JSON type information is being sent.

Create a JSON document using StaticJsonDocument<256> doc.

Prepare role of entry as doc[“role”], granted of entry as doc[“granted”] and timestamp of entry as doc[“timestamp”].

Use serializeJson(doc, body) to format the JSON object to text. Use .POST() to send the body to API and store the response from the Server in code. End the communication using .end().

Return true if successfully logged. Else, false. (Checked using (code==200 || code==201)).

14. bool queueLog(LogEntry &entry) :

entry is referenced using ‘&’ to avoid copying of the entry (String variables) that might occupy RAM.

Log the queue if offline queue isn’t full yet (checked using (queueCount<MAX_QUEUE)).

Store the information of entry in offlineQueue and increment queueCount.